

Introduction to Transformers

Naeemullah Khan

naeemullah.khan@kaust.edu.sa

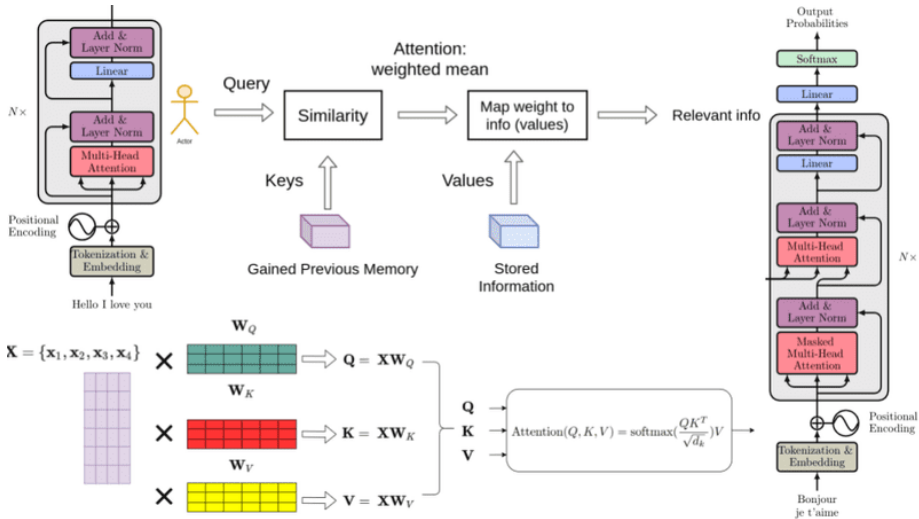


جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology



LMH
Lady Margaret Hall
University of Oxford

June 12, 2026



1. Learning Outcomes
2. Introduction to Transformers
3. Self-Attention Mechanism
4. Multi-Head Attention
5. Encoder-Decoder Architecture
6. Casual Masking
7. Pre-LayerNorm vs. Post-LayerNorm Architectures
8. Layer Normalization vs RMSNorm
9. Transformer Variants
10. Limitations of Transformers
11. Future Directions

► Limited Context:

- RNNs process sequences step-by-step → Difficult to capture long-range dependencies.
- CNNs are effective for local patterns → Struggle with global context in sequences.

► Sequential Bottlenecks:

- RNNs process data sequentially.
- Prevents efficient parallelization.
- Leads to slow training and inference.

► Slow Training:

- RNNs and CNNs are computationally intensive.
- Especially challenging for long sequences or large images.
- Results in increased training times.

There is a need for models that:

- ▶ Capture comprehensive, global context efficiently.
- ▶ Allow for parallel computation.
- ▶ Accelerate training and inference.

Transformers address these needs by:

- ▶ Leveraging self-attention mechanisms.
- ▶ Modeling dependencies across entire sequences.
- ▶ Enabling highly parallelizable computation.

After this lecture, you will be able to:

- ▶ Explain the motivation behind the development of Transformers.
- ▶ Describe the architecture and key components of Transformer models.
- ▶ Understand and implement attention and self-attention mechanisms.
- ▶ Explain the role of positional encoding in Transformers.
- ▶ Distinguish between general attention and self-attention.
- ▶ Understand the concept and benefits of multi-head attention.
- ▶ Compare CNNs with and without self-attention.
- ▶ Summarize the advantages of Transformers over traditional sequence models.

After this session, you should be able to:

- ▶ Understand the self-attention mechanism
- ▶ Learn how multi-head attention works and why it helps
- ▶ Grasp the role of positional encoding
- ▶ Deconstruct transformer blocks and variants
- ▶ Compare Pre-LayerNorm vs. Post-LayerNorm
- ▶ Evaluate LayerNorm vs. RMSNorm
- ▶ Explore causal masking in decoder-only transformers
- ▶ Understand tradeoffs and future directions

► What is a Transformer?

- Introduced by Vaswani et al. (2017) – “Attention is All You Need”

► Key features:

- No recurrence, all attention
- Encoder-decoder structure
- Enables parallel processing
- Scales well with data and compute

- ▶ Computes contextual representation of a word using all other words
- ▶ **Inputs:** Query (Q), Key (K), Value (V)
- ▶ **Output:** Weighted sum of values

Attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- ▶ Enables each word to attend to all others

- ▶ **Query:** What we're looking for (e.g., the meaning of "bank" in context)
- ▶ **Keys:** Tags or features associated with each word/sentence (e.g., "river", "money")
- ▶ **Values:** Information linked to each key (e.g., context-specific meaning)
- ▶ Uses dot-product similarity between Query and Keys to assign attention weights
- ▶ Weighted sum of Values produces the contextual representation

- ▶ Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions.
- ▶ It consists of multiple attention heads, each with its own set of parameters.
- ▶ The outputs of all heads are concatenated and linearly transformed.

Formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

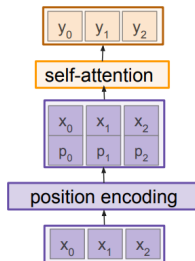
where each head is defined as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- ▶ W_i^Q , W_i^K , and W_i^V are learned projection matrices for each head.
- ▶ W^O is the output projection matrix that combines the outputs of all heads.

- ▶ Enables the model to jointly attend to information from different representation subspaces at different positions.
- ▶ Captures various linguistic and syntactic patterns by using multiple attention heads.
- ▶ Improves the model's ability to represent complex relationships in the data.
- ▶ Provides richer and more diverse feature representations.

- ▶ **Positional Encoding:** Supplies sequence order information to models that lack recurrence or convolution, such as Transformers.
- ▶ Concatenate or add a special positional encoding p_j to each input vector x_j .
- ▶ A function $\text{pos} : \mathbb{N} \rightarrow \mathbb{R}^D$ maps the position j of the vector into a D -dimensional vector, i.e., $p_j = \text{pos}(j)$.

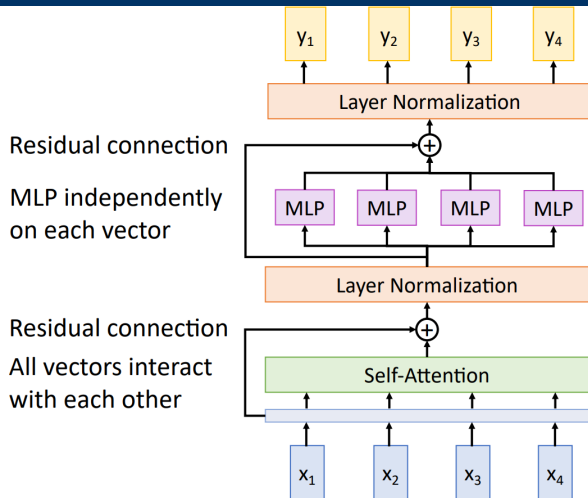


- ▶ The positional encoding is added to the input vectors before computing the attention scores.
- ▶ This allows the model to incorporate information about the position of each vector in the sequence.
- ▶ The positional encoding can be learned or fixed, depending on the implementation.
- ▶ The choice of positional encoding can affect the model's ability to capture long-range dependencies and relationships in the data.
- ▶ **Sinusoidal encodings:** Common fixed positional encoding uses sine and cosine functions:

$$\text{pos}(j)_{2k} = \sin\left(\frac{j}{10000^{2k/d}}\right)$$
$$\text{pos}(j)_{2k+1} = \cos\left(\frac{j}{10000^{2k/d}}\right)$$

Attention is all you need

Vaswani et al, NeurIPS 2017



- **Input:** A set of vectors $\mathbf{x}$, one per token.
- **Output:** A set of vectors $\mathbf{y}$, one per token.
- **Self-Attention:** Allows each token to attend to all others, capturing contextual relationships.
- **Layer Normalization & MLP:** Applied independently to each token, ensuring stability and expressiveness.
- **Key Properties:** Highly scalable and parallelizable due to independent operations across tokens.

Figure 2: Detailed view of a single Transformer block.

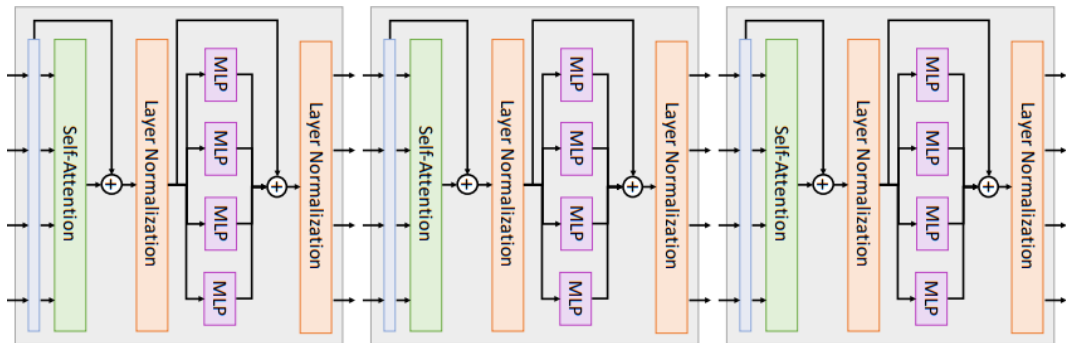


Figure 3: Stacking multiple Transformer blocks.

- A Transformer consists of a sequence of identical blocks, each refining the token representations.

- ▶ In the original architecture (Vaswani et al., 2017): 12 blocks, hidden size $D_Q = 512$, and 6 attention heads were used.
- ▶ The modular design enables deep stacking and efficient parallel computation.

- ▶ **Encoder:** Stack of self-attention and feed-forward (FF) layers.
- ▶ **Decoder:** Adds masked self-attention and encoder-decoder attention on top of FF layers.
- ▶ **Applications:** Widely used in tasks like translation, summarization, and more.

- ▶ **Example:** BERT
- ▶ Only the encoder stack is used.
- ▶ **Applications:** Classification, question answering, embeddings.

- ▶ **Example:** GPT, LLaMA
- ▶ Uses only the decoder block.
- ▶ **Masked self-attention (causal):** Each token can only attend to previous tokens.
- ▶ **Applications:** Language modeling, text generation.

- ▶ **Purpose:** Prevents the model from attending to future tokens in the sequence.
- ▶ **Implementation:** Uses a triangular mask in the self-attention mechanism.
- ▶ **Effect:** Ensures that predictions for a token only depend on previous tokens, maintaining the autoregressive property.

Feature	Pre-LN	Post-LN
Stability	✓ More stable	✗ Less stable
Gradient flow	✓ Better	✗ Can explode/vanish
Used in	GPT-3, T5	Original Transformer

Table 1: Comparison of Pre-LayerNorm and Post-LayerNorm Architectures

Pre-LN: LayerNorm applied *before* sublayers

Post-LN: LayerNorm applied *after* sublayers

LayerNorm: Normalizes across the feature dimension by subtracting the mean and dividing by the standard deviation.

RMSNorm: Removes the mean and scales by the root mean square (RMS) of the features.

- ▶ Simpler
- ▶ Faster
- ▶ Slightly better in large-scale settings **zhou2022rmsnorm**

- ▶ **Encoder-only:** BERT, RoBERTa
- ▶ **Decoder-only:** GPT, LLaMA, Falcon
- ▶ **Encoder-Decoder:** T5, BART, mT5

Modifications:

- ▶ **Long-range:** Longformer, BigBird
- ▶ **Efficient:** Linformer, Performer
- ▶ **Sparse:** Sparse Transformer, Reformer

- ▶ High compute and memory requirements
- ▶ Poor extrapolation capabilities
- ▶ Limitations due to hard-coded positional encoding
- ▶ Quadratic attention cost ($O(n^2)$)
- ▶ Lack of reasoning and grounding

- ▶ Linear and sparse attention mechanisms
- ▶ Memory-efficient transformers (e.g., FlashAttention)
- ▶ Integration with retrieval, reasoning, and tool use
- ▶ Multimodal transformers (e.g., Flamingo, GPT-4V)
- ▶ Biologically inspired architectures (e.g., RWKV, State Space Models)

- ▶ Transformers use self-attention to capture relationships within sequences.
- ▶ Self-attention allows each token to attend to all other tokens, enabling context-aware representations.
- ▶ Positional encoding is crucial for maintaining the order of tokens in the sequence.
- ▶ Masked self-attention prevents future information leakage during training.
- ▶ Multi-head attention enhances model capacity by allowing multiple attention mechanisms to learn different aspects of the data.

- [1] Fei-Fei Li, Yunzhu Li, and Ruohan Gao. *Stanford CS231n: Deep Learning for Computer Vision*. <http://cs231n.stanford.edu/index.html>
- [2] Assaf Shocher, Shai Bagon, Meirav Galun, and Tali Dekel. *WAIC DL4CV Deep Learning for Computer Vision: Fundamentals and Applications*.
<https://dl4cv.github.io/index.html>
- [3] Justin Johnson. *UMich EECS 498.008/598.008: Deep Learning for Computer Vision*.
<https://web.eecs.umich.edu/~justincj/teaching/eecs498/WI2022/>
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017. <https://arxiv.org/abs/1706.03762>
- [5] Stanford CS231n. *Lecture 11: Attention & Transformers*.
<http://cs231n.stanford.edu/slides/2023/lecture11.pdf>

- [6] Carnegie Mellon University. *Deep Learning Lecture: Vision Transformers and Transformers*.
<https://deeplearning.cs.cmu.edu/F23/index.html>
<https://www.youtube.com/watch?v=1gq2y1E5g6M>

- [7] MIT 6.S191. *Introduction to Deep Learning Resources*.
<https://introtodeeplearning.com/>

- [8] Satya Mallick. *LearnOpenCV: A Comprehensive Guide to Attention Mechanisms in Deep Learning*. <https://learnopencv.com/attention-mechanism-in-neural-networks/>